Sr.	Problem Definition
1	WAP to find weather given number is **Harshad number or not**. A Harshad number (or Niven number) is an integer that can be divided evenly by the sum of its own digits. For example, 18 is a Harshad number because sum of digit is (1+8) =9 which perfectly divides the number 18 (18%9==0)
2	WAP to find weather given number is **Ugly or not**. An **ugly number** is a *positive* integer which does not have a prime factor other than 2, 3, and 5. **Input**: n = 6 **Output**: true **Explanation**: 6 = 2 × 3
3	WAP to enter an element at specific position into **array**. **(Do not take a new array)** **Input**: Enter size of array: 5 Enter elements: 10 20 30 40 50 Enter element to insert: 25 Enter position: 3 **Output**: 10 20 25 30 40 50
4	WAP to Convert a **Decimal to Hexa-decimal** and vice versa.
5	WAP to find weather given number is **Pronic or not**. **A Pronic Number** is defined as a number that is the product of two consecutive non-negative integers. In other words, N is a Pronic Number if there exists a non-negative integer k such that N = k * (k + 1). E.g. **6 is a Pronic Number** because 6=2*3
6	Write a program to check whether number is **Happy number or not**. A **happy number** are those number whose digit's square summation eventually reaches to 1, if the sequence start repeating then it is not a happy number. E.g., **49 is a happy number** whose sequence is 49 97 130 10 1. E.g., **50 is not a happy number** whose sequence is 50 25 29 85 89 145 42 20 4 16 37 58 89 it should stop when 89 is detected 2nd time and print 50 is not a happy number.
7	WAP to implement a following **pattern** 1 1 * 1 1 * 3 * 1 1 * 3 * 5 * 3 * 1
8	Given two integer arrays **nums1 and nums2, return an array of their intersection**. Each element in the result must appear as many times as it shows in both arrays and you may return the result in sorted order.
9	WAP to find weather given number is **Automorphic or not**. An **automorphic** number is a number whose square ends with the same digits as the number itself. For example, **5 is automorphic** because 5^2 = 25, which ends in 5. Similarly, **76 is automorphic** because 76^2 = 5776, which ends in 76.

<table>
<tr><th>Sr.</th><th>Problem Definition</th></tr>
<tr><td>10</td><td>WAP to sort an Array using insertion sort.</td></tr>
<tr><td>11</td><td>Implement a following pattern
<table>
<tr><td>For n=4
1
2 5
3 6 8
4 7 9 10</td><td>For n=5
1
2 6
3 7 10
4 8 11 13
5 9 12 14 15</td></tr>
</table></td></tr>
<tr><td>12</td><td>Given an array, rotate the array to the right by k steps, where k is non-negative.
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]</td></tr>
<tr><td>13</td><td>Implement a following pattern
* * * * * * *
* *
* *
*
* *
* *
* * * * * * *</td></tr>
<tr><td>14</td><td>Write a program that should decode the given pattern and print the resulting expanded string.
Input: 2a3bc4dE5F2G7H
Output: aabbbcddddEFFFFFGGHHHHHHH</td></tr>
<tr><td>15</td><td>Given an array arr[], the task is to reverse the array. Reversing an array means rearranging the elements such that the first element becomes the last, the second element becomes second last and so on. (Do not use any additional Array)
Input: arr[] = {1, 4, 3, 2, 6, 5}
Output: {5, 6, 2, 3, 4, 1}</td></tr>
<tr><td>16</td><td>WAP for following Scenario.
Given n rupees and a chocolate price of m for each chocolate, with a wrapper exchange offer of 1 chocolate per k wrappers, calculate the total number of chocolates you can eat with n rupees.</td></tr>
<tr><td>17</td><td>Given a string that contains a special character together with alphabets (‘a’ to ‘z’ and ‘A’ to ‘Z’), reverse the string in a way that special characters are not affected.
Input: a!!!b.c.d,e'f,ghi
Output: i!!!h.g.f,e'd,cba
Input: str = “Ab,c,de!$”
Output: str = “ed,c,bA!$”</td></tr>
<tr><td>18</td><td>Write a C program to replace every duplicate occurrence of an element with 0, while keeping the first occurrence unchanged.</td></tr>
</table>

Sr.	Problem Definition
	• Do not use a second array. • The first occurrence of every element must remain unchanged. • Every subsequent occurrence should be replaced by 0. • The original order of elements should be maintained. **Input**: Enter size of array: 7 Enter elements: 4 2 7 4 2 9 7 **Output**: 4 2 7 0 0 9 0
19	Count Elements Greater Than All Previous Elements Write a C program to count how many elements in an array are **greater than every element that appears before them**. • The **first element is always counted**, because there are no elements before it. • An element is counted only if it is **strictly greater** than all previous elements. • Do not sort the array. **Input:** Enter size of array: 7 Enter elements: 3 5 2 8 6 10 9 **Output**: Count = 4
20	Write a C program to find the **smallest positive integer that is not present** in the given integer array. • The array may contain **positive, negative, and zero** values. • The answer must be a **positive integer**. • Duplicate elements may be present. **Input**: Enter size of array: 5 Enter elements: 3 4 -1 1 2 **Output**: Smallest missing positive number = 5
21	Write a C program to **remove all occurrences of a given element from an array without changing the order of the remaining elements**. **Conditions:** • Do not use a second array. • The relative order of the remaining elements must remain unchanged. • Display the resulting array and its new size. **Input:** Enter size of array: 7 Enter elements: 10 20 30 20 40 20 50 Enter element to remove: 20 **Output**: 10 30 40 50